Assessment 3 Machine Learning Lab

Apurba Koirala

22BCE3799

Guided by: Dr. Sudha S

1. Write a program to construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using standard Heart Disease Data Set. Calculate the accuracy, precision, and recall for your data set.

```
! pip install --upgrade pgmpy
! pip install pgmpy.models pgmpy.estimators pgmpy.inference
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score,
classification_report, confusion_matrix

from pgmpy.estimators import HillClimbSearch, BIC, BayesianEstimator
from pgmpy.models import DiscreteBayesianNetwork
```

```
target = "target"
for col in data.columns:
    if col != target and data[col].dtype != "object":
        data[col] = pd.qcut(data[col], q=4, duplicates="drop",
labels=False).astype(str)
```

```
data[target] = data[target].astype(str)
```

```
train, test = train_test_split(data, test_size=0.2, random_state=42)
```

```
hc = HillClimbSearch(train)
best_model = hc.estimate(scoring_method=BIC(train))
```

```
model = DiscreteBayesianNetwork(best_model.edges())
```

```
model.fit(train, estimator=BayesianEstimator, prior_type="BDeu")
```

```
# Keep only the model's nodes
predictors = list(set(model.nodes()) - {target})
y_pred = model.predict(test[predictors])[target].tolist()
y_true = test[target].tolist()
```

```
print("Predictions:", len(y_pred), "out of", len(test))
print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred, average="weighted"))
print("Recall:", recall_score(y_true, y_pred, average="weighted"))
print("\nClassification Report:\n", classification_report(y_true, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_true, y_pred))
```

```
Predictions: 205 out of 205
Accuracy: 0.5658536585365853
Precision: 0.5659072071820375
Recall: 0.5658536585365853

Classification Report:
              precision    recall  f1-score   support

           0       0.57      0.54      0.55       102
           1       0.56      0.59      0.58       103

    accuracy                           0.57       205
   macro avg       0.57      0.57      0.57       205
weighted avg       0.57      0.57      0.57       205


Confusion Matrix:
 [[55 47]
 [42 61]]
```

```
print("Nodes in model:", model.nodes())
print("Target in model?", target in model.nodes())
```

```
Nodes in model: ['sex', 'target', 'cp', 'exang', 'fbs', 'ca', 'thal', 'slope', 'oldpeak', 'restecg']
Target in model? True
```

2. Implement K Nearest Neighbors algorithm for iris dataset and evaluate the algorithm with accuracy, precision, recall and F1-score.

```
from sklearn.datasets import load_iris

iris = load_iris()
```

```
X = iris.data
y = iris.target
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
```

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
```

```
y_pred = knn.predict(X_test)
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-score: {f1:.4f}")
```

```
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-score: 1.0000
```

3. Develop and fine-tune a Multi-Layer Perceptron (MLP) neural network model with back-propagation using scikit-learn, perform hyper parameter tuning, and assess its performance using various performance measures for a specific machine learning task.

```
from sklearn.datasets import load_breast_cancer

breast_cancer = load_breast_cancer()

print(breast_cancer.DESCR)
```

```
import pandas as pd
from sklearn.preprocessing import StandardScaler


df = pd.DataFrame(breast_cancer.data, columns=breast_cancer.feature_names)


df['target'] = breast_cancer.target


print("Missing values before handling:")
display(df.isnull().sum())


X = df.drop('target', axis=1)
y = df['target']


scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)


X_scaled_df = pd.DataFrame(X_scaled, columns=breast_cancer.feature_names)

print("\nScaled features:")
display(X_scaled_df.head())
```

```
from sklearn.model_selection import train_test_split


X_train, X_test, y_train, y_test = train_test_split(X_scaled_df, y,
test_size=0.2, random_state=42)

print("Shape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)
```

```
from sklearn.neural_network import MLPClassifier

mlp_model = MLPClassifier(hidden_layer_sizes=(100,), activation='relu',
solver='adam', random_state=42)
```

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'hidden_layer_sizes': [(50,), (100,), (50, 50), (100, 50)],
    'activation': ['tanh', 'relu'],
    'solver': ['sgd', 'adam'],
    'alpha': [0.0001, 0.001, 0.01]
}

grid_search = GridSearchCV(mlp_model, param_grid, cv=5, n_jobs=-1)

grid_search.fit(X_train, y_train)

best_params = grid_search.best_params_

print("Best hyperparameters found:")
print(best_params)
```

```
Best hyperparameters found:
{'activation': 'relu', 'alpha': 0.0001, 'hidden_layer_sizes': (50,), 'solver': 'adam'}
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:
  warnings.warn(
```

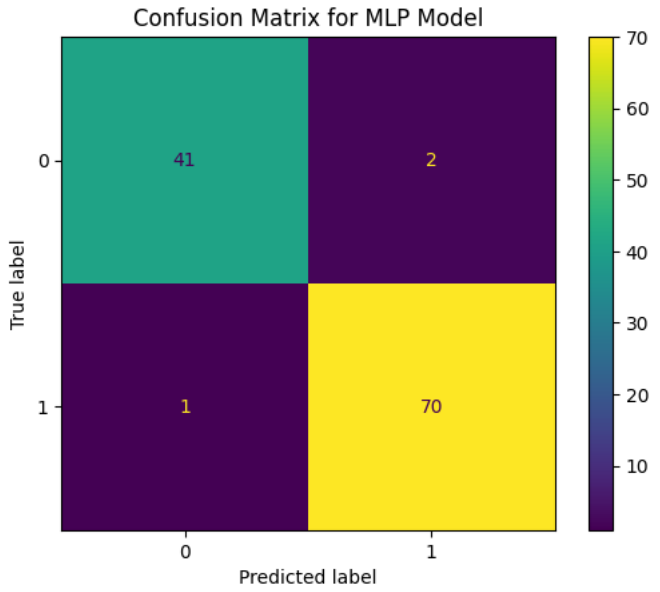
```
from sklearn.neural_network import MLPClassifier

best_mlp_model = MLPClassifier(**best_params, random_state=42)

best_mlp_model.fit(X_train, y_train)
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score

y_pred = best_mlp_model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

```
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-score: {f1:.4f}")
```

```
Accuracy: 0.9737
Precision: 0.9722
Recall: 0.9859
F1-score: 0.9790
```

```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title('Confusion Matrix for MLP Model')
plt.show()
```

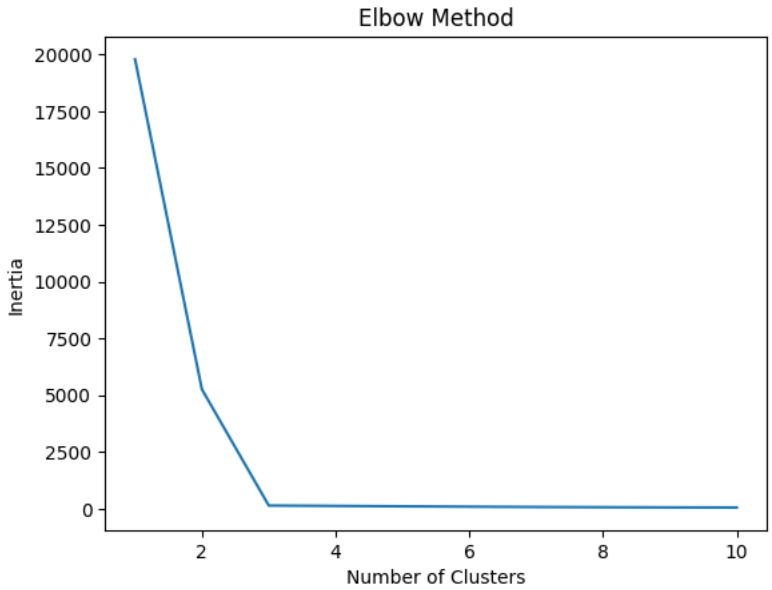


4. Illustrate the K-means clustering to cluster the data points for at least five epochs properly.
   a. Use the elbow method to determine the optimal number of clusters.
   b. Visualize the clusters.
   c. Plot the centroids of each cluster.

```
from sklearn.datasets import make_blobs
```

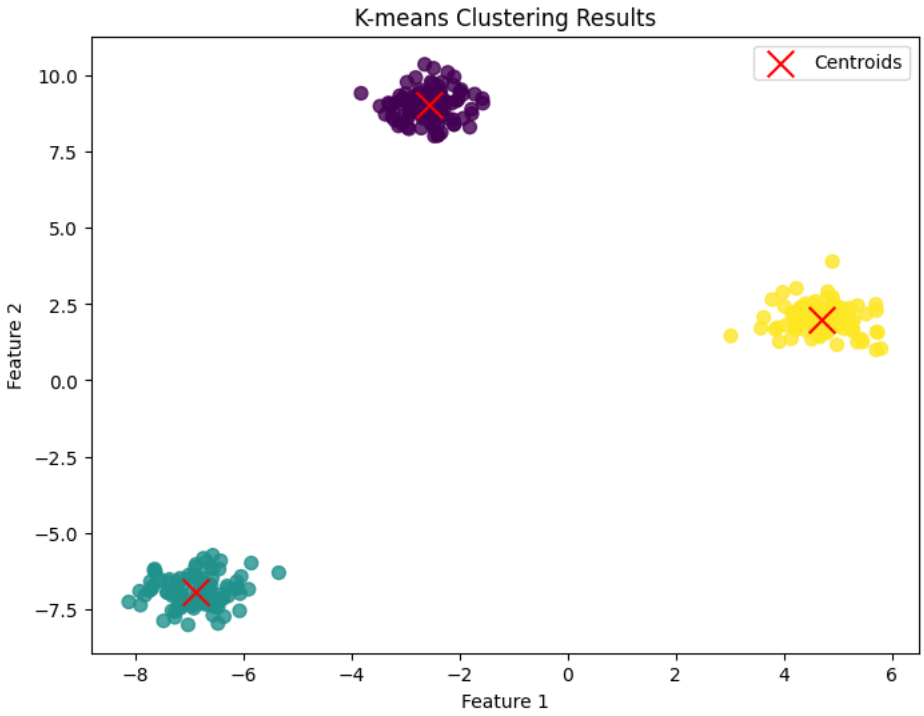
```
X, _ = make_blobs(n_samples=300, centers=3, cluster_std=0.5,
random_state=42)
print(X.shape)
```

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

inertia = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)

plt.plot(range(1, 11), inertia)
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
```




```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X)
labels = kmeans.labels_
centroids = kmeans.cluster_centers_
```

```
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=50, alpha=0.8)
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', marker='x', s=200,
label='Centroids')
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("K-means Clustering Results")
plt.legend()
plt.show()
```

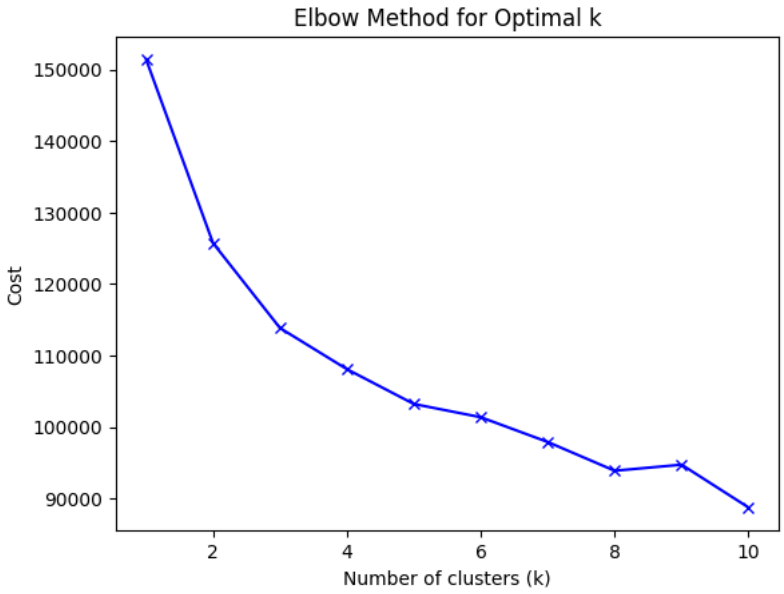


5. Illustrate the K-mode clustering to cluster the bank customers.

```
print(df.dtypes)
print(df.isnull().sum())

categorical_cols = ['job', 'marital', 'education', 'default', 'housing',
'loan', 'contact', 'month', 'poutcome', 'term_deposit']
df_categorical = df[categorical_cols]

display(df_categorical.head())
```

```
!pip install kmodes
```

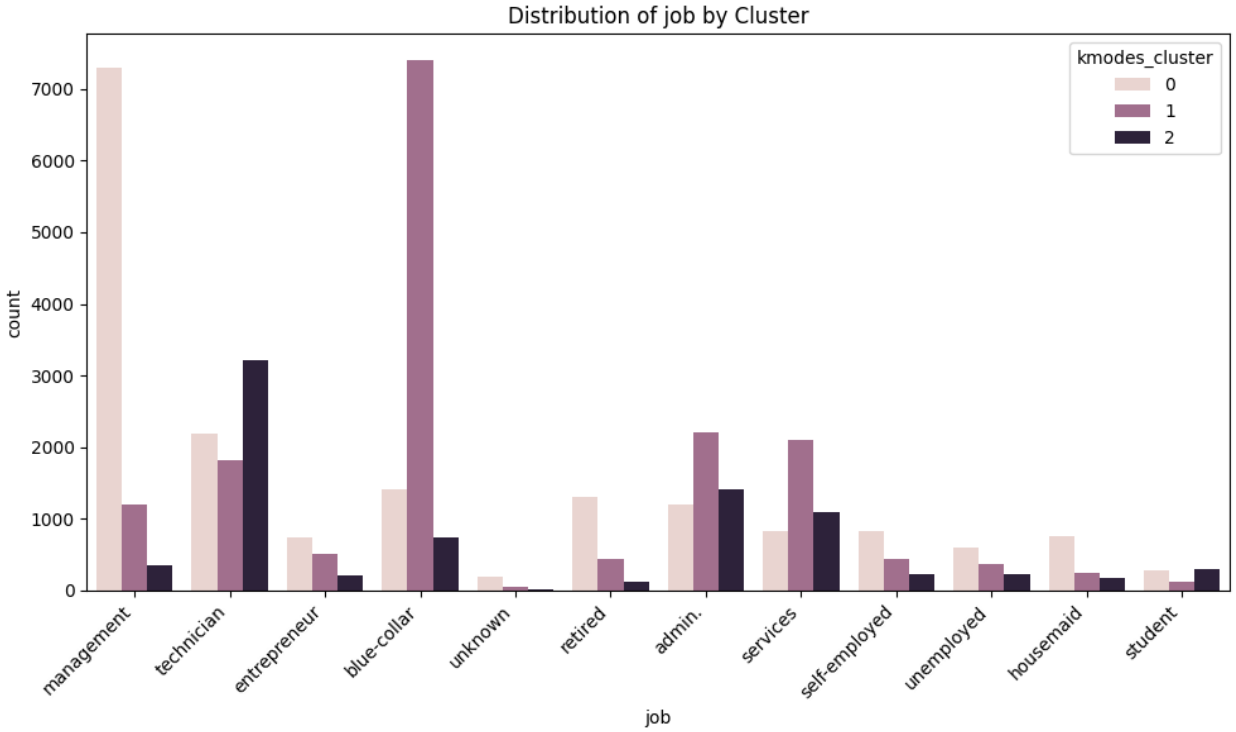
```
from kmodes.kmodes import KModes
import matplotlib.pyplot as plt
costs = []
K = range(1, 11)
for k in K:
    kmode = KModes(n_clusters=k, init='Huang', n_init=5, random_state=0)
    kmode.fit_predict(df_categorical)
    costs.append(kmode.cost_)

plt.plot(K, costs, 'bx-')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Cost')
plt.title('Elbow Method for Optimal k')
plt.show()
```

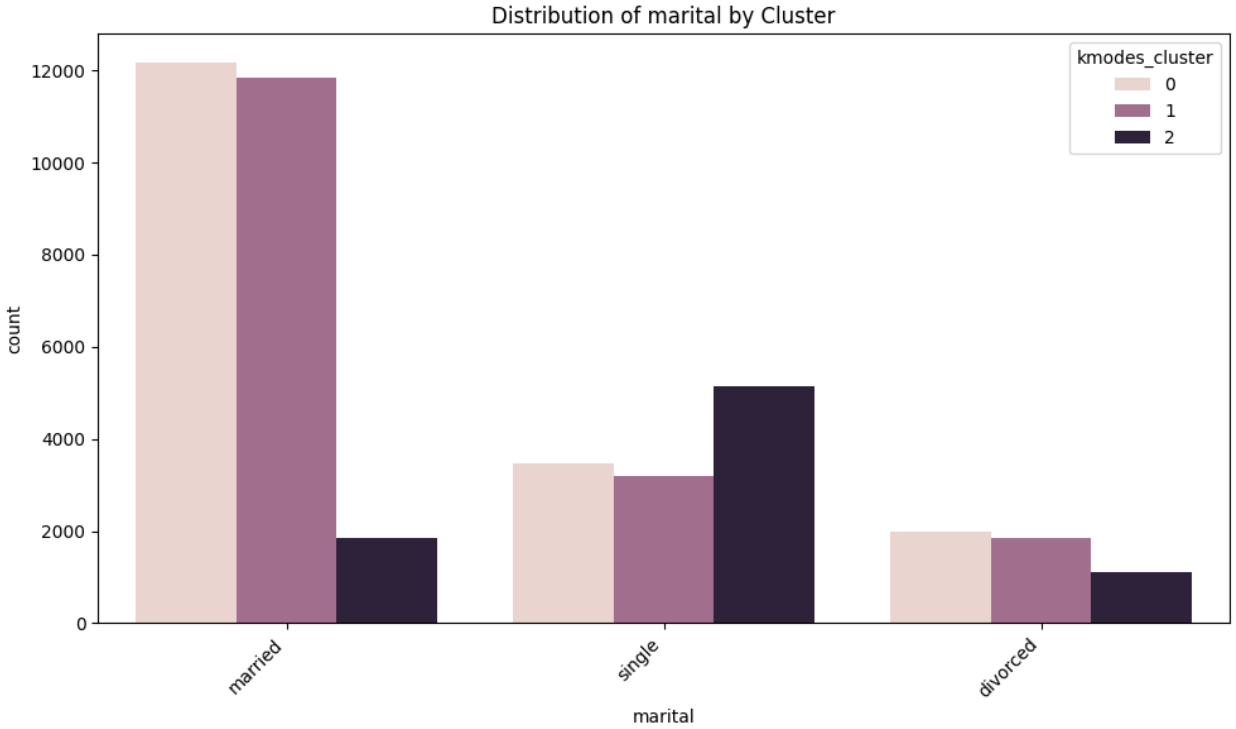


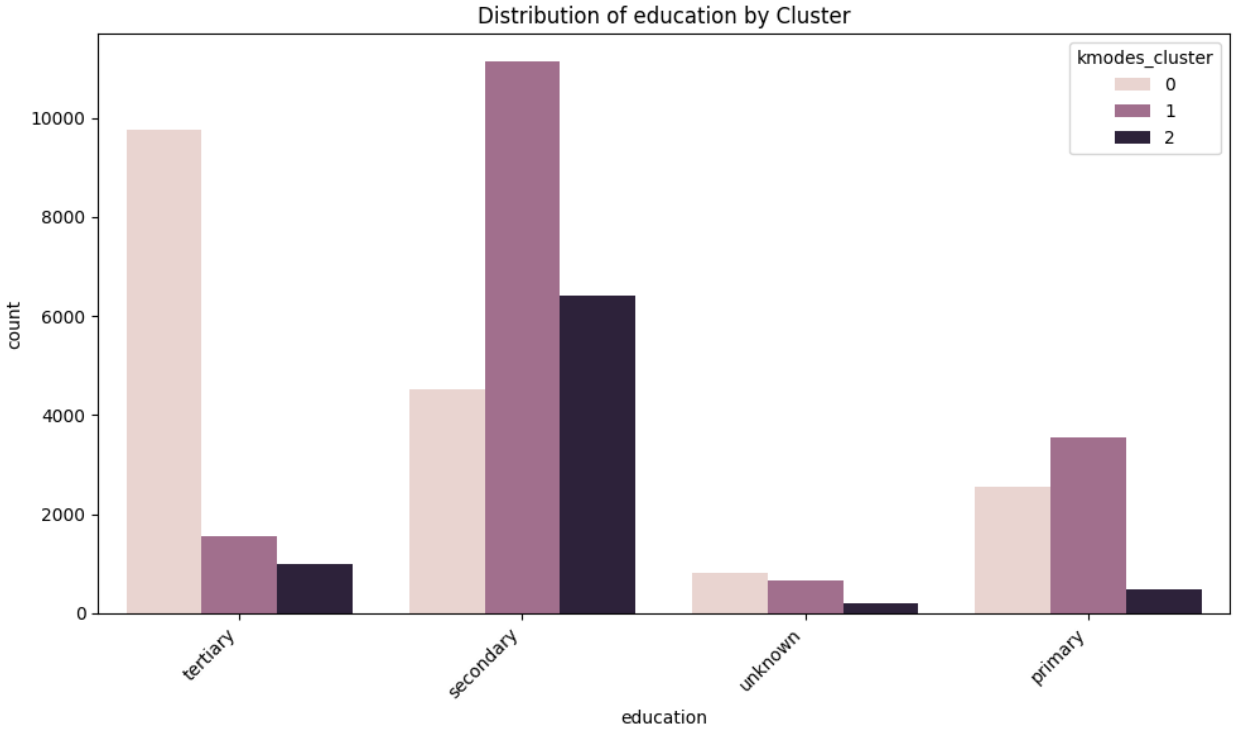

```
optimal_k = 3
kmode = KModes(n_clusters=optimal_k, init='Huang', n_init=5,
random_state=0)
clusters = kmode.fit_predict(df_categorical)
df['kmodes_cluster'] = clusters

display(df.head())
```

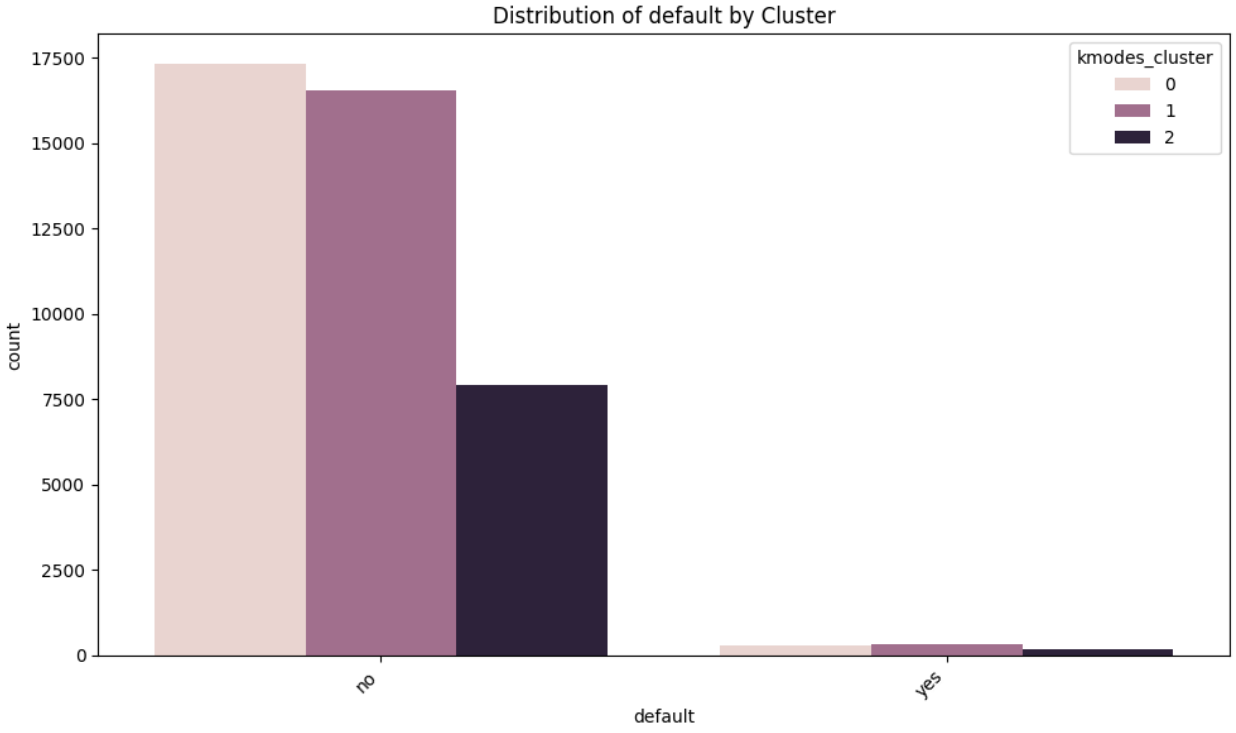
	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	term_deposit	kmodes_cluster
0	58	management	married	tertiary	no	2143	yes	no	unknown	5	may	261	1	-1	0	unknown	no	1
1	44	technician	single	secondary	no	29	yes	no	unknown	5	may	151	1	-1	0	unknown	no	1
2	33	entrepreneur	married	secondary	no	2	yes	yes	unknown	5	may	76	1	-1	0	unknown	no	1
3	47	blue-collar	married	unknown	no	1506	yes	no	unknown	5	may	92	1	-1	0	unknown	no	1
4	33	unknown	single	unknown	no	1	no	no	unknown	5	may	198	1	-1	0	unknown	no	1

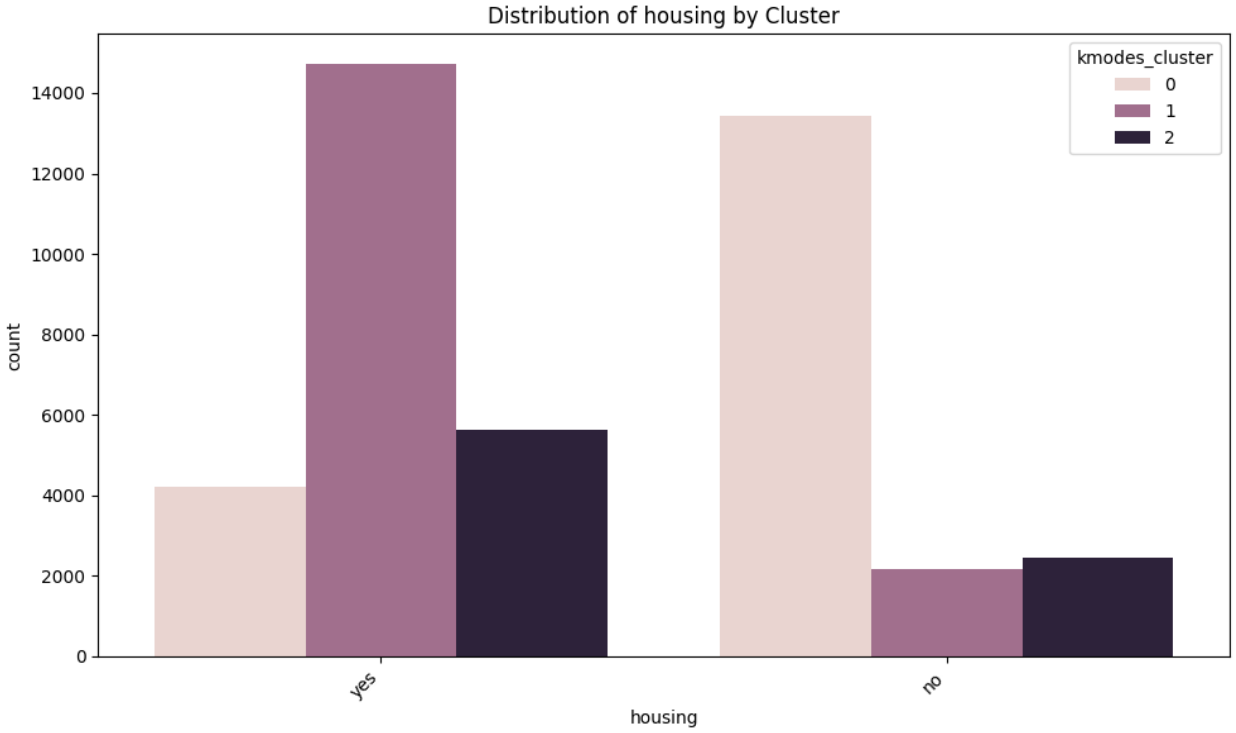
```
import seaborn as sns

for col in categorical_cols:
    plt.figure(figsize=(10, 6))
    sns.countplot(data=df, x=col, hue='kmodes_cluster')
    plt.title(f'Distribution of {col} by Cluster')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
```

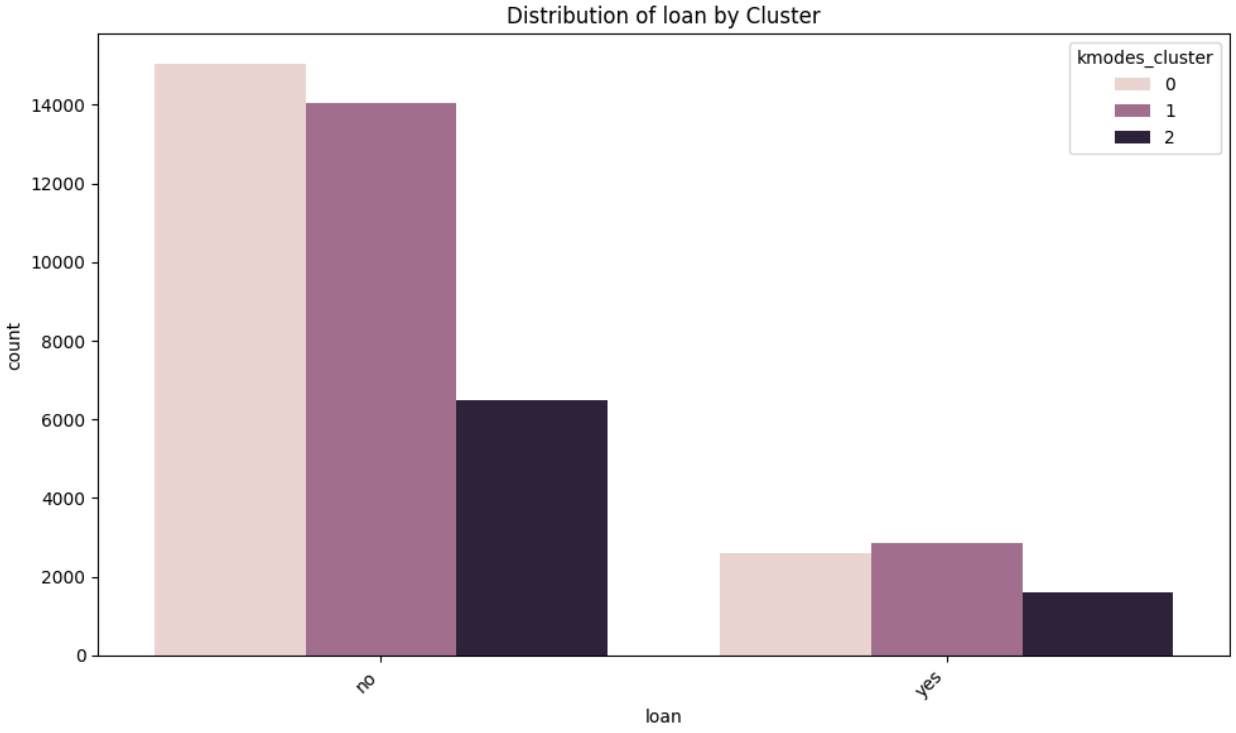

Distribution of job by Cluster
kmodes_cluster
0
1
2
count
0
1000
2000
3000
4000
5000
6000
7000
management
technician
entrepreneur
blue-collar
unknown
retired
admin.
services
self-employed
unemployed
housemaid
student
job

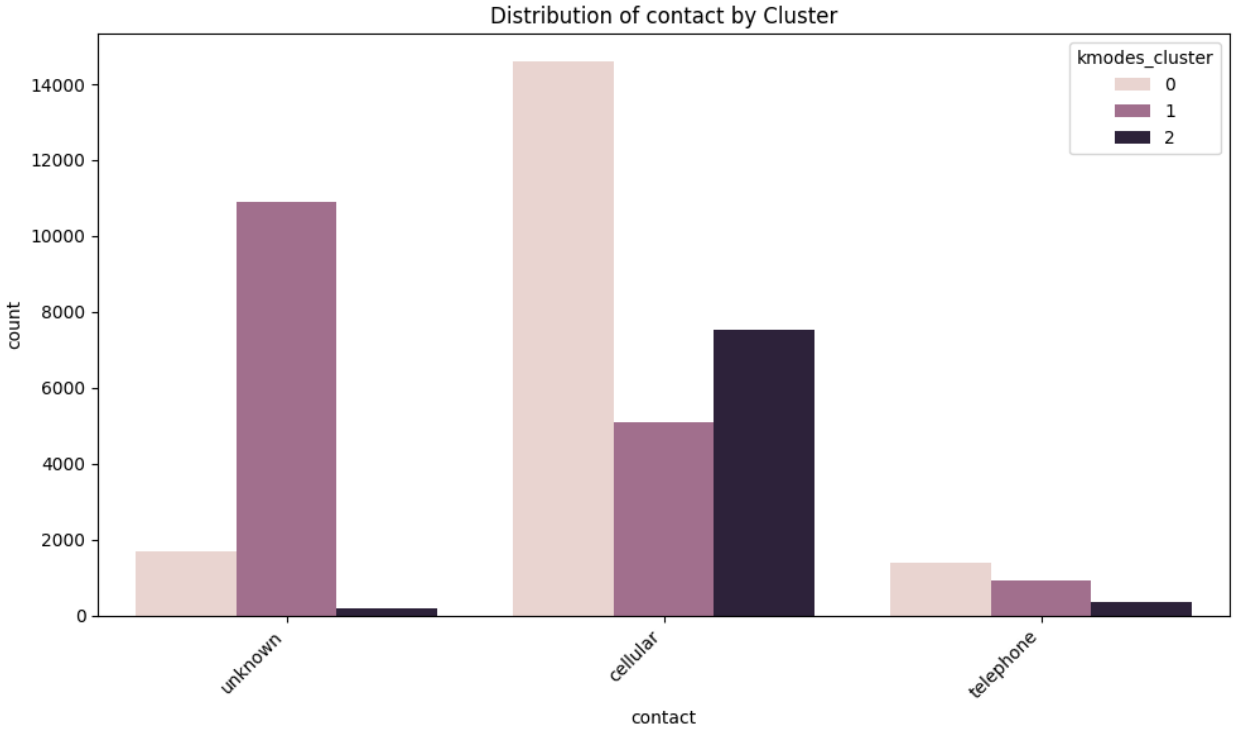

Distribution of marital by Cluster
kmodes_cluster
0
1
2
count
0
2000
4000
6000
8000
10000
12000
married
single
divorced
marital

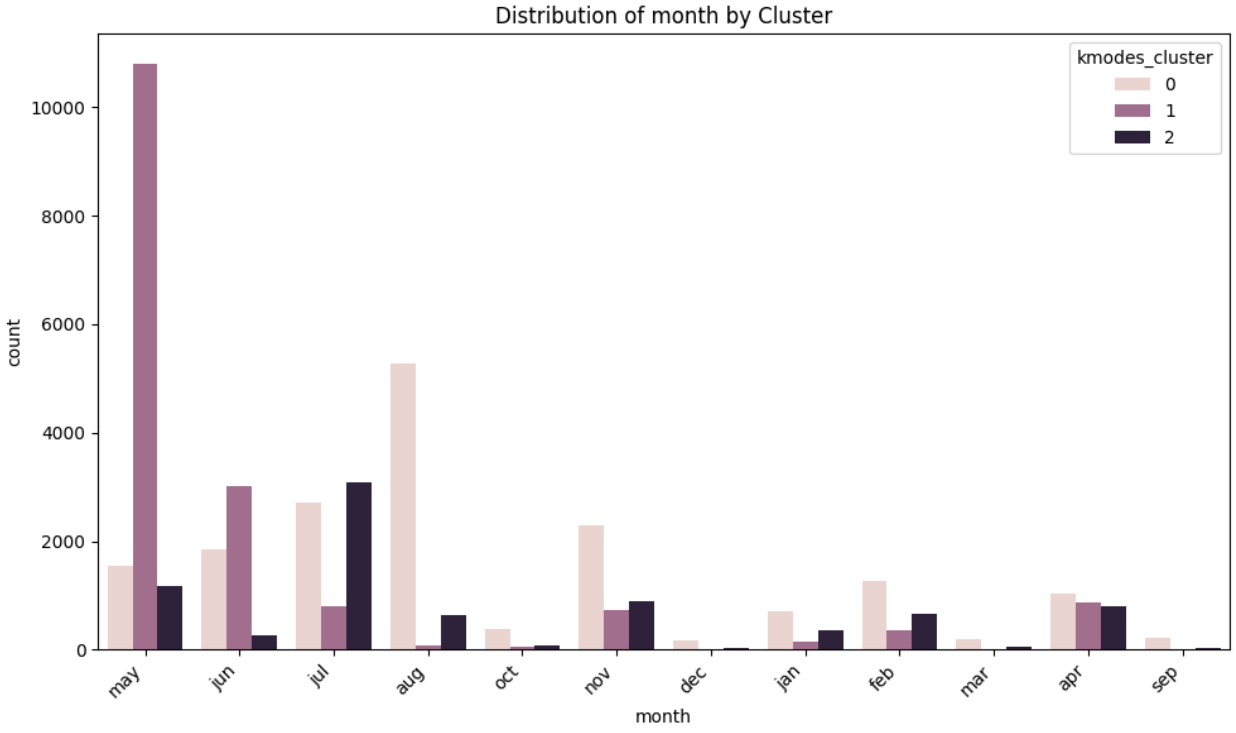

Distribution of education by Cluster
kmodes_cluster
0
1
2
count
10000
8000
6000
4000
2000
0
tertiary
secondary
unknown
primary
education

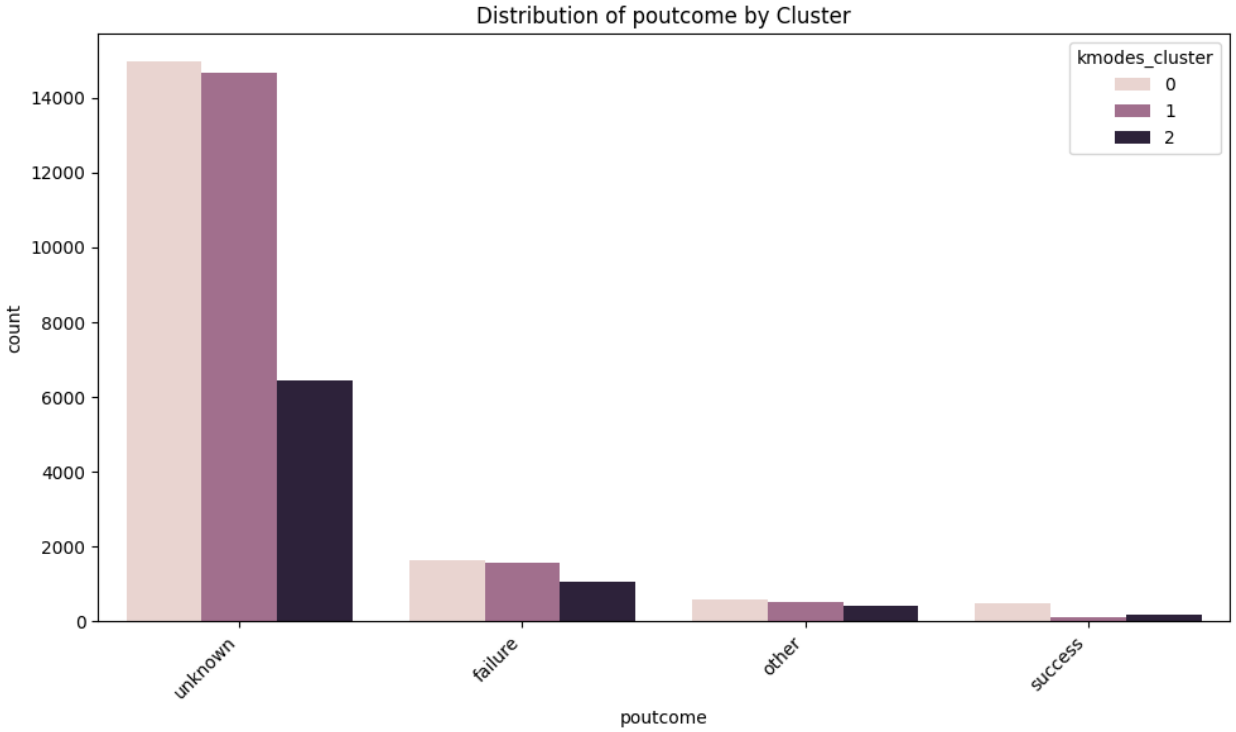

Distribution of default by Cluster
kmodes_cluster
0
1
2
count
17500
15000
12500
10000
7500
5000
2500
0
no
yes
default

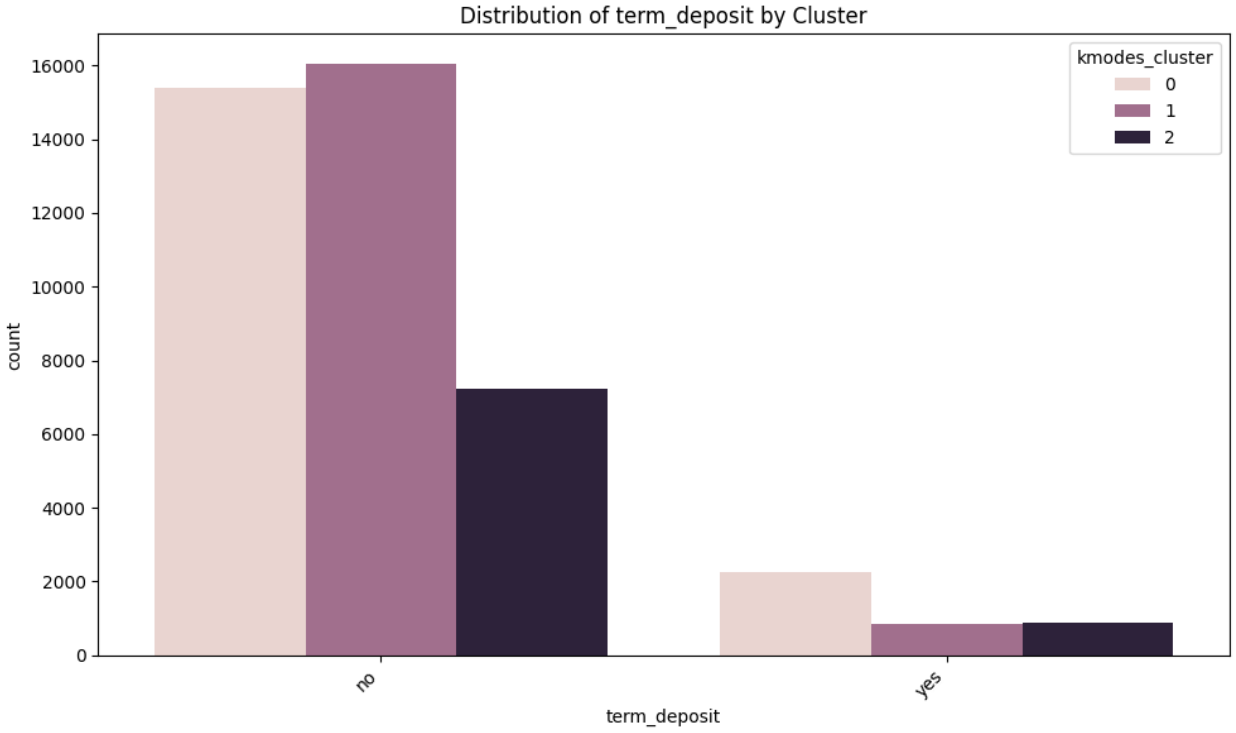

Distribution of housing by Cluster
kmodes_cluster
0
1
2
14000
12000
10000
8000
6000
4000
2000
0
count
yes
no
housing


Distribution of loan by Cluster
kmodes_cluster
0
1
2
14000
12000
10000
8000
6000
4000
2000
0
count
no
yes
loan


Distribution of contact by Cluster
kmodes_cluster
0
1
2
14000
12000
10000
8000
6000
4000
2000
0
count
unknown
cellular
telephone
contact


Distribution of month by Cluster
kmodes_cluster
0
1
2
10000
8000
6000
4000
2000
0
count
may
jun
jul
aug
oct
nov
dec
jan
feb
mar
apr
sep
month


Distribution of poutcome by Cluster
kmodes_cluster
0
1
2
count
0
2000
4000
6000
8000
10000
12000
14000
unknown
failure
other
success
poutcome


Distribution of term_deposit by Cluster
kmodes_cluster
0
1
2
count
0
2000
4000
6000
8000
10000
12000
14000
16000
no
yes
term_deposit